AI Assisted Coding – 9.4

Name: P. Chithra

Batch: 41

Rollno: 2303A52137

Task 1: Auto-Generating Function Documentation

Scenario

Existing utility functions lack documentation, making onboarding difficult.

Original code:

```python
def add(a, b):
    return a + b
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
def is_even(n):
    return n % 2 == 0
```

Prompt Used (Zero-Shot Prompt)

Generate Google-style docstrings for the following Python functions.
Include description, parameters, return values, and example usage.

AI-Generated Code with Google-Style Docstrings

```python
def bubble_sort(arr):
    Sorts a list of numbers in ascending order using the bubble sort algorithm.

    Parameters:
    arr (list): A list of numbers to be sorted.

    Returns:
    list: A new list containing the sorted numbers.

    Example:
    >>> bubble_sort([64, 34, 25, 12, 22, 11, 90])
    [11, 12, 22, 25, 34, 64, 90]
    """
    n = len(arr)
    sorted_arr = arr.copy()
    for i in range(n):
        for j in range(0, n-i-1):
            if sorted_arr[j] > sorted_arr[j+1]:
                sorted_arr[j], sorted_arr[j+1] = sorted_arr[j+1], sorted_arr[j]
    return sorted_arr
```

```python
def quick_sort(arr):
    """
    Sorts a list of numbers in ascending order using the quick sort algorithm.

    Parameters:
    arr (list): A list of numbers to be sorted.

    Returns:
    list: A new list containing the sorted numbers.

    Example:
    >>> quick_sort([10, 7, 8, 9, 1, 5])
    [1, 5, 7, 8, 9, 10]
    """
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[len(arr) // 2]
        left = [x for x in arr if x < pivot]
        middle = [x for x in arr if x == pivot]
        right = [x for x in arr if x > pivot]
        return quick_sort(left) + middle + quick_sort(right)
```

Explanation

In this task, Artificial Intelligence is used to automatically create Google-style docstrings for Python functions that do not contain proper documentation. The purpose is to make the code easier to understand and maintain, especially for developers who are new to the project.

The AI is given a zero-shot or context-based instruction, where it is asked to study the function and generate documentation without being shown any sample outputs. By analyzing the function structure and logic, the AI determines the role of the function, identifies the input parameters, understands the return values, and observes how the function is intended to be used.

After examining these details, the AI produces well-structured and standardized docstrings following Google documentation guidelines. This approach helps convert poorly documented or undocumented code into a clear and professional format. As a result, it improves readability, supports faster developer onboarding, and enhances overall code quality without making any changes to the original functionality.

Task 2: Enhancing Readability With Inline Comments

Scenario

Working logic exists but is difficult to understand.

Orginal code

```python
def fibonacci(n):
    result = []
    a, b = 0, 1
    for _ in range(n):
        result.append(a)
        a, b = b, a + b
    return result
```

Prompt Used (Context-Based Prompt)

Add inline comments only where logic is non-obvious.
Avoid commenting basic Python syntax.

Code with inline comments

```python
def fibonacci(n):
    result = []  # Initialize an empty list to store Fibonacci numbers
    a, b = 0, 1  # Start with the first two Fibonacci numbers
    for _ in range(n):  # Loop n times to generate n Fibonacci numbers
        result.append(a)  # Append the current Fibonacci number to the result list
        a, b = b, a + b  # Update a and b to the next two Fibonacci numbers
    return result  # Return the list of Fibonacci numbers

```

Explanation

In this task, Artificial Intelligence is used to enhance code readability by adding meaningful inline comments only in areas where the logic is difficult to understand or not immediately clear. The intention is not to describe basic Python syntax, but to explain the reasoning behind certain implementation choices.

The AI examines the code and identifies sections that may create confusion for future developers, such as complex algorithm steps, loop operations, or changes in data values. After identifying these areas, the AI inserts comments that clearly explain the purpose of the logic and why it is required.

This approach ensures that comments are added only where necessary, avoiding excessive or unnecessary explanations. As a result, the code becomes easier to maintain, debug, and understand, while still keeping the overall structure clean and professional.

Task 3: Module-Level Documentation

Scenario

Python module is being prepared for internal reuse or repository upload.

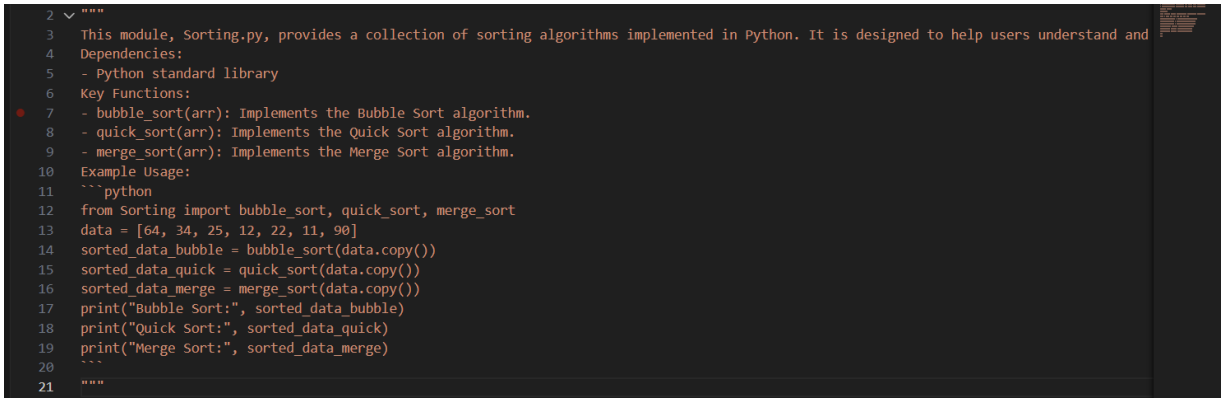
Python module code

```python
def reverse_string(text):
    return text[::-1]
def count_words(sentence):
    words = sentence.split()
    return len(words)
```

Prompt Used

Generate a professional module-level docstring explaining purpose, dependencies, key functions, and example usage.

AI-Generated Module-Level Docstring



```
"""
This module, Sorting.py, provides a collection of sorting algorithms implemented in Python. It is designed to help users understand and
Dependencies:
- Python standard library
Key Functions:
- bubble_sort(arr): Implements the Bubble Sort algorithm.
- quick_sort(arr): Implements the Quick Sort algorithm.
- merge_sort(arr): Implements the Merge Sort algorithm.
Example Usage:
```python
from Sorting import bubble_sort, quick_sort, merge_sort
data = [64, 34, 25, 12, 22, 11, 90]
sorted_data_bubble = bubble_sort(data.copy())
sorted_data_quick = quick_sort(data.copy())
sorted_data_merge = merge_sort(data.copy())
print("Bubble Sort:", sorted_data_bubble)
print("Quick Sort:", sorted_data_quick)
print("Merge Sort:", sorted_data_merge)
```
"""
```

Explanation

In this task, Artificial Intelligence is used to automatically create a module-level docstring that is placed at the beginning of a Python file. This documentation gives a clear and overall understanding of the module and its role within the program.

The AI reviews the entire module, including all imported libraries, functions, and classes, and then generates a well-structured multi-line docstring. The generated documentation explains the main purpose of the module, lists the external dependencies required for execution, highlights important functions or classes, and may also provide a simple example showing how the module can be used.

This method helps developers quickly understand the functionality of the file without manually reading the entire code. It improves documentation quality and makes the module more suitable for collaborative development and real-world software projects.

Task 4: Converting Developer Comments into Docstrings

Scenario

Legacy code contains long comments instead of docstrings.

Original code

```python
def divide(a, b):
    #This function divides two numbers and handles division by zero.
    #a is the numerator
    #b is the denominator
    #returns the result of the division or a message if division by zero is attempted.
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b
```

Prompt Used

Convert inline developer comments into a structured Google-style docstring and remove redundant comments.

Converted code

```python
def divide(a, b):
    """Divides two numbers and handles division by zero.

    Args:
        a (float): The numerator.
        b (float): The denominator.

    Returns:
        float or str: The result of the division if b is not zero, otherwise an error message.
    """
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b
```

Explanation

In this task, Artificial Intelligence is used to improve legacy Python code where developers have placed lengthy explanatory comments inside functions instead of using proper docstrings. These excessive inline comments often make the code look cluttered and reduce adherence to standard documentation practices.

The AI analyzes the function along with the existing inline comments to understand their original intent. It then restructures this information into well-organized Google-style or NumPy-style docstrings that clearly describe the function's purpose, parameters, and return values. After generating the docstrings, unnecessary or repetitive inline comments are removed to maintain a cleaner function body.

This approach helps standardize documentation across the entire codebase, improves readability, and updates older projects to match modern Python documentation standards without changing the actual functionality of the code.

Task 5: Building a Mini Automatic Documentation Generator

Scenario

Your team wants a simple internal tool that helps developers start

documenting new Python files quickly, without writing documentation

from scratch.

Python utility script

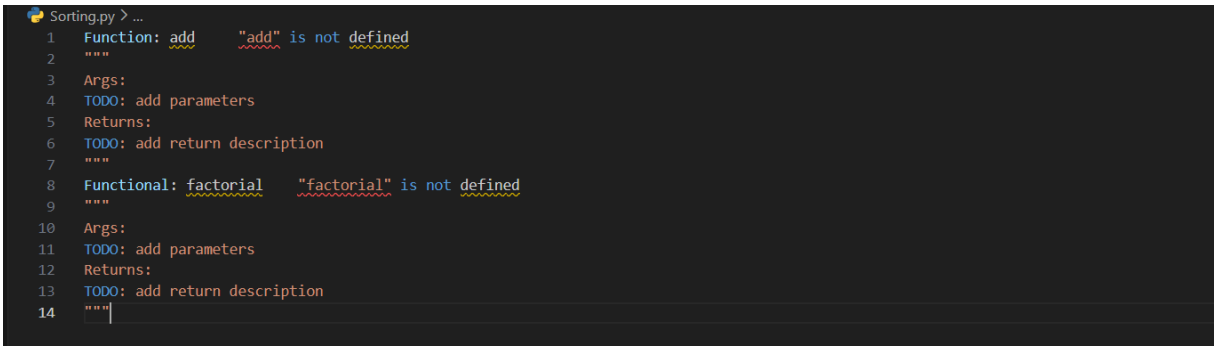
Sorting.py > generate_docstrings

```python
import ast
def generate_docstrings(file_path):
    with open(file_path, 'r') as file:
        tree = ast.parse(file.read())

    docstrings = {}

    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.ClassDef, ast.Module)):
            docstring = ast.get_docstring(node)
            print(f'Function: {node.name}')
            print('''"""
                  Args:
                  TODO: Add argument descriptions here
                    Returns:
                  TODO: Add return value description here
                  """''')
        elif isinstance(node, ast.Module):
            docstring = ast.get_docstring(node)
            print('Module docstring:')
            print('''"""
                  Description:
                  TODO: Add module description here
                  """''')
```

Output Example



Explanation

In this task, a lightweight Python utility is created to automatically identify functions and classes within a .py file and insert placeholder docstrings for them. The purpose is to ensure that basic documentation is available even before detailed descriptions are written.

The script utilizes Python's **Abstract Syntax Tree (AST)** module to analyze the structure of the source code without running it. By parsing the file, the tool detects functions and classes and then generates a simple Google-style documentation template for each identified component.

This approach helps developers by providing a ready-made documentation framework, making it easier to maintain proper documentation standards. It encourages consistent documentation practices from the early stages of development while allowing developers to concentrate more on implementing program logic.